NRLC-09: Toolchain Reference & End-to-End Runbook

> **Series:** NRLC — New Relic to Dynatrace Migration Deep Dives | **Notebook:** 9 of 9 | **Created:** April 2026 | **Last Updated:** 10/31/2026

Overview

The master reference for the open-source toolchain that powers every preceding notebook in this series. This deep dive covers how `Dynatrace-NewRelic`, `nrql-engine`, and `nrql-translator` fit together, how to operate the orchestrating CLI, the configuration surface, the export formats (Monaco v2 YAML, Terraform HCL), and a complete end-to-end runbook.

As of 2026-04-15 the Python orchestrator ships **40+ transformers** across 9 phase batches (Phase 11 foundation + Phases 16–24 expansion), **1,200+ unit tests**, and **292 NRQL translation patterns** pinned to the TypeScript `nrql-engine` via the Phase 19b regression suite. It emits **Gen3 Dynatrace by default** (Workflows, anomaly detectors, Segments, OpenPipeline, Document API); use `--legacy` only for the 8 Gen2-only capabilities or for tenants without Gen3 features.

Table of Contents

1. The Three Projects — How They Fit
2. Installation & Configuration
3. `migrate.py` CLI Reference
4. Component Selection
5. Export Formats — Monaco & Terraform
6. End-to-End Runbook
7. CI/CD Integration Patterns

Prerequisites

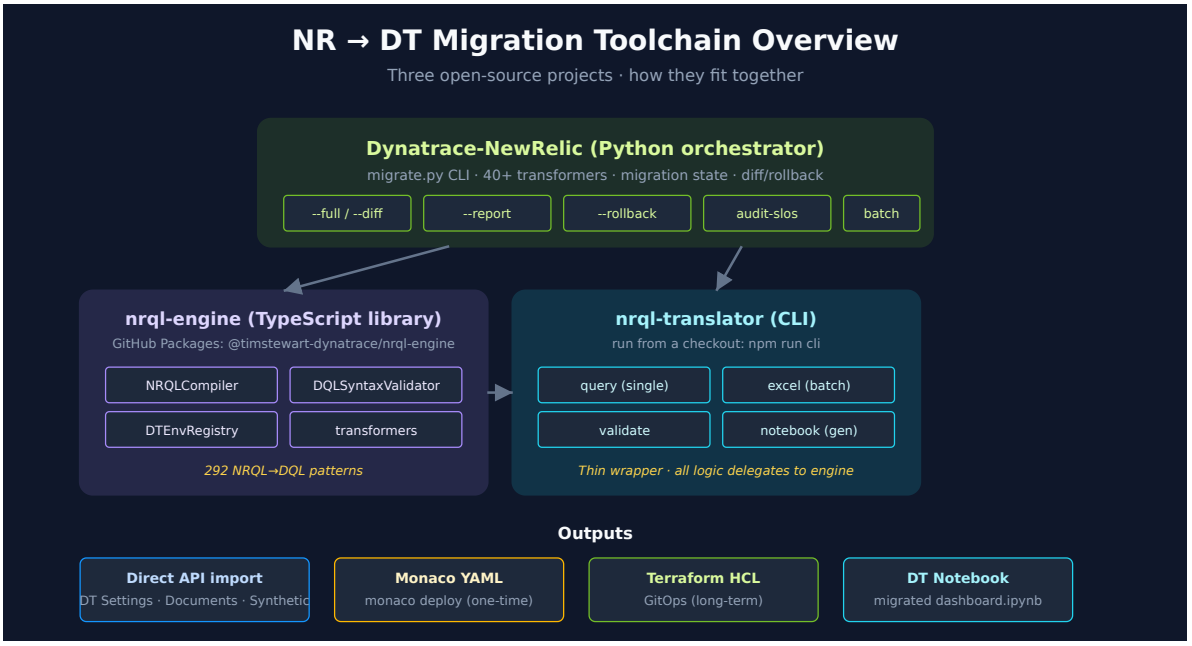
Requirement	Details
Audience	Engineers operating the migration end-to-end
Required reading	All preceding NRLC notebooks (this is the operational reference)
Access	NR API key + DT API token; both with appropriate scopes (see §2)

> For the full Terraform resource catalog (worked examples for every resource named in §5's export list, plus the OAuth-client-only vs. Platform-Token distinction for IAM/bucket resources), see **AUTOM-04** (Terraform Provider) and **AUTOM-02** (Settings API) rather than treating this notebook as the canonical shape.

1. The Three Projects — How They Fit



Use Case	Tool
End-to-end NR→DT migration	`Dynatrace-NewRelic` (`migrate.py`)
Embed translation in your own tool/web app	`nrql-engine`
One-off NRQL→DQL on the command line	`nrql-translator`
Translate a batch of dashboard queries	`nrql-translator excel` or `migrate.py batch`
Generate a DT notebook from a `*_ast_test.json` file	`nrql-translator notebook`



Upstream Home: Dynatrace Migration Assistant (`dynatrace-dma`)

The official Dynatrace migration tooling lives under the `dynatrace-dma` GitHub organization — the **Dynatrace Migration Assistant** project. Existing repos in the org:

Repo	Source Platform	Status
`splunk-to-dynatrace`	Splunk	Active
`datadog-to-dynatrace`	Datadog	Active
`newrelic-to-dynatrace` *(planned)*	New Relic	`nrql-engine` planned to relocate here

When the New Relic engine relocates, the recommended install path will become the `dynatrace-dma` repo. Until then, `timstewart-dynatrace/nrql-engine` is canonical.

2. Installation & Configuration

Dynatrace-NewRelic (Python)

```
git clone https://github.com/timstewart-dynatrace/NewRelic-to-Dynatrace-Migration-Utilities
cd NewRelic-to-Dynatrace-Migration-Utilities
pip install -r requirements.txt
cp .env.example .env
# Edit .env with NR + DT credentials
```

> **Future home note:** The `nrql-engine` underlying this orchestrator is planned to relocate to the `dynatrace-dma` (**Dynatrace Migration Assistant**) organization, joining the existing `splunk-to-dynatrace` and `datadog-to-dynatrace` repos. Track that org for the official Dynatrace-supported install going forward.

Required NR API Token Scopes

- `read:dashboards`, `read:alerts`, `read:nrql`, `read:synthetics`, `read:slo`, `read:workloads`, `read:logs`

Required DT API Token Scopes

From the tool's token scopes reference (migration utilities GitHub):

Credential	Permissions
Platform token (`dt0s16.`, the default): preflight	`settings:schemas:read`, `settings:objects:read`, `document:documents:read`, `automation:workflows:read`, `slo:slos:read`
Platform token: full migrate adds	`settings:objects:write`, `document:documents:write`, `automation:workflows:write`, `automation:workflows:run`, `slo:slos:write`
Platform token: Grail reads	`storage:logs:read`, `storage:events:read`, `storage:metrics:read`, `storage:spans:read`, `storage:entities:read`, `storage:buckets:read`
Classic token (`dt0c01.`, with `--legacy`)	`ReadConfig`, `WriteConfig`, `DataExport`, `ExternalSyntheticIntegration`, `events.ingest`

> **Note:** OAuth clients use the same platform permissions — not the classic `<area>.<action>` scopes. SLOs are platform-only here: the tool writes them through the SLO Service Public API, so the credential needs `slo:slos:read` (plus `slo:slos:write` for migrate runs), not `slo.read` / `slo.write`; the Terraform `dynatrace_platform_slo` resource takes an OAuth client (`DT_CLIENT_ID`, `DT_CLIENT_SECRET`, `DT_ACCOUNT_ID`) with those permissions.

Configuration File (`.env`)

Key names as in the repository's `.env.example` (`main` branch, read 10/05/2026):

```
NEW_RELIC_API_KEY=NRAK-...
NEW_RELIC_ACCOUNT_ID=1234567
NEW_RELIC_REGION=US           # or EU

DYNATRACE_API_TOKEN=dt0c01.XXXX...
DYNATRACE_ENVIRONMENT_URL=https://abc12345.live.dynatrace.com
DYNATRACE_DETECTOR_ACTOR=     # service-user UUID; anomaly detectors fail to import without it

# Optional
MIGRATION_DRY_RUN=false
MIGRATION_OUTPUT_DIR=./output
LOG_LEVEL=INFO
```

`audit-slos` additionally reads `DYNATRACE_OAUTH_TOKEN` from the environment and exits without it.

Settings are loaded via Pydantic; missing required values fail fast with descriptive errors.

3. `migrate.py` CLI Reference

Subcommand inventory (post-Phase-24)

The `migrate.py` entry point registers **14 subcommands** (`main` branch, read 10/05/2026):

`agents, archive, audit, audit-slos, batch, compile, convert, export-monaco, export-terraform, extract-nrql, migrate, preflight, reference, scan-instrumentation`

Subcommand	Purpose
`migrate`	Full pipeline (export → transform → import). Accepts `--full`, `--export-only`, `--import-only`, `--dry-run`, `--components`, `--rollback <file>`, `--retry <file>`, `--resume`, `--incremental`, `--report`, `--diff`, `--legacy`, `--canary <pct>`, `--canary-auto-proceed`
`compile`	Single NRQL → DQL compilation. Flags: `--interactive`, `--file`, `--validate`, `--output`
`convert`	NRQL → DQL with post-processing + auto-fix (wraps `compile` + `DQLFixer`)
`batch`	CSV / Excel batch compile (NRQL column)
`reference`	Print NRQL → DQL reference table; `--mappings` for full mapping tables (230 metrics + 72 attributes + 90+ aggregations + 34 event types)
`preflight`	**Phase 14.** Probe target DT tenant for Gen3 API availability (Settings 2.0 / Document / Automation). Suggests `--legacy` if any surface is missing.
`agents`	**Phase 16.** Per-language APM agent migration action plans (Java / .NET / Node.js / Python / Ruby / PHP / Go). Flags: `--language <lang>`, `--phase`, `--dry-run`.
`scan-instrumentation`	**Phase 16.** Scan **one source file** (`--file`) for `newrelic.*()` SDK calls; print DT/OTel replacement suggestions as a diff (`--output` writes it to a file). Side-effect-free; manual apply.
`archive`	**Phase 17.** Pre-decommission NRDB snapshot (resumable JSONL per event type). Requires `--account` and `--since`.
`audit`	**Phase 20.** Config drift: compares a transformed baseline (`--baseline <dynatrace_config.json>`) with the live tenant. Reports RENAMED / DELETED / MODIFIED / EXTRA. Read-only; exits 1 on drift. Does not query New Relic.
`audit-slos`	Validate DT SLOs against live metrics for missing/invalid keys (Phase 11 + Phase 20 enrichment). DT side only; needs `DYNATRACE_OAUTH_TOKEN`.
`extract-nrql`	Extract every NRQL query from the export JSON into a flat `.txt` or `.csv` list.
`export-monaco`	Emit Monaco v2 project YAML (Gen3 default; `--legacy` for Gen2 shapes).
`export-terraform`	Emit Terraform HCL with `dynatrace-oss/dynatrace` provider (Gen3 default; `--legacy` for Gen2 shapes).

> **Note:** No subcommand runs NRQL against New Relic and DQL against Dynatrace to compare results. Behavioral validation

> (NRLC-08 §4) is a manual step; `audit` compares *configuration*, not query results.

`migrate` flags

These are options of the `migrate` subcommand, not global flags. There is no `--transform-only`, `--filter`, `--log-level` or `--no-color`.

Flag	Purpose
`--components <list>`	Restrict to specific components (see §4 for the accepted names)
`--export-only` / `--import-only`	Run one phase only; `--import-only` needs `--input <dir>` and writes no rollback manifest
`--dry-run`	Export and transform, skip the import ("Validate without applying changes")
`--diff`	Compare the transformed entities with the live tenant — use with `--dry-run`, otherwise it runs after the import
`--output <dir>`	Override output directory
`--rollback <file>`	Delete the entities listed in a rollback manifest
`--legacy`	Gen2 output path — emits Alerting Profiles / Management Zones / Auto-Tags / Config v1 dashboards instead of Gen3 equivalents.
`--canary <pct>`	Phase 20. Two-wave import — import N% first, await approval, then import remainder
`--canary-auto-proceed`	In canary mode, skip the interactive prompt and proceed automatically (CI / scripted use)
`--report`	Emit enriched conversion report (confidence_score, warning_codes, runbook_url per entry)

Transformer coverage

`migrate.py` orchestrates 40+ entity transformers across 9 phase batches. See the coverage matrix (migration utilities GitHub) for the complete NR-surface → transformer mapping. High-level summary:

Phase	Transformers added
11 (foundation)	alert, dashboard, synthetic, slo, workload, infrastructure, log_parsing, tag, drop_rule
16 (P0 coverage)	agents/ orchestrators (7 languages), lambda, browser_rum, mobile_rum, custom_instrumentation_translator
17 (P1 alerts + data + identity)	non_nrql_alert, baseline_alert, lookup_table, maintenance_window, change_tracking, custom_event_ingest, identity, log_obfuscation, tools/nrdb_archive
18 (specialized products)	cloud_integration, kubernetes, aiops, vulnerability, npm, ai_monitoring, prometheus
19 (dashboard widget parity)	funnel / honeycomb / event-feed / cascading vars / saved views / permissions uplift
19b (nrql-engine parity)	compiler/shorthands, K8s metric + entity-field overrides, 24 DQL fixer methods
20 (operational safety)	migration/canary, migration/audit, DynatraceClient.delete_entity, enriched ConversionReport
23 (second-wave parity)	key_transaction, otel_metrics, statsd, cloudwatch_metric_streams, metric_transform plugin, transformers/mappings/
24 (third-wave parity)	database_monitoring (10 engines), on_host_integration (12 techs), security_signals, custom_entity, log_archive, metric_normalization, synthetic_specialized (cert-check + broken-links), saved_filter_notebook, otel_collector, legacy/error_inbox_v1, legacy/request_naming_v1

4. Component Selection

Migrating one entity type at a time is the safe pattern. The `--components` flag accepts a comma-separated list.

> **What `migrate --components` actually accepts (`main` branch, read 10/05/2026).** `--list-components` prints ten names: `dashboards`, `alerts`, `synthetics`, `slos`, `workloads`, `notification_channels`, `infrastructure`, `log_parsing`, `tags`, `drop_rules`. Of those, `migrate` exports and transforms only `dashboards`, `alerts`, `notification_channels`, `synthetics`, `slos` and `workloads`; any other name is reported as exported and produces nothing. Dependencies are added automatically — `alerts` pulls in `notification_channels`, and `slos` pulls in `alerts`. The table below maps NR surfaces to the engine's **transformers**; most of its first-column names are not `--components` values.

Component	Migrates	Transformer
`dashboards`	NR Dashboards → DT Documents (one per page), including funnel / honeycomb / event-feed / cascading variables / saved views	`dashboard_transformer` (Phase 11 + Phase 19 widget parity)
`alerts`	NR NRQL conditions → `builtin:davis.anomaly-detectors` + Workflow	`alert_transformer` (Phase 11 + Phase 17 extensions)
`baseline_alerts`	NR baseline / outlier NRQL conditions → Dynatrace Intelligence adaptive detectors	`baseline_alert_transformer` (Phase 17)
`non_nrql_alerts`	Infrastructure / Synthetic / Browser / Mobile / External-service / Multi-location-synthetic conditions	`non_nrql_alert_transformer` (Phase 17)
`lookup_tables`	NR `WHERE IN` lookups → Resource Store JSONL + DQL `lookup` subquery	`lookup_table_transformer` (Phase 17)
`maintenance_windows`	NR scheduled + recurring maintenance + mute rules → DT maintenance-window envelopes; a mute rule's condition arrives as a LOW-confidence comment in a workflow filter	`maintenance_window_transformer` (Phase 17)
`change_tracking`	NR deployment markers → DT events API (`CUSTOM_DEPLOYMENT`, `CUSTOM_CONFIGURATION`)	`change_tracking_transformer` (Phase 17)
`custom_event_ingest`	NR custom events → bizevent CloudEvent payloads	`custom_event_ingest_transformer` (Phase 17)
`synthetics`	NR Ping / Browser / Scripted API / Scripted Browser → `dynatrace_http_monitor` / `dynatrace_browser_monitor` (Config-v1, `ExternalSyntheticIntegration` scope)	`synthetic_transformer` (Phase 11)
`synthetics_specialized`	CERT_CHECK + BROKEN_LINKS	`synthetic_specialized_transformer` (Phase 24)
`slos`	NR SLOs (v1 / v2) → Platform SLOs (SLO Service Public API; `dynatrace_platform_slo`). Classic `builtin:monitoring.slo` is no longer emitted	`slo_transformer` (Phase 11)
`key_transactions`	NR Key Transactions → SLO + OpenPipeline enrichment + Workflow bundle	`key_transaction_transformer` (Phase 23)
`workloads`	NR Workloads → Grail Filter Segment (`dynatrace_segment`) + bucket-scoped IAM	`workload_transformer` (Phase 11, Gen3 default)
`tags`	NR tag rules → OpenPipeline enrichment (`builtin:openpipeline.*`)	`tag_transformer` (Phase 11, Gen3 default)
`logs`	NR log forwarding configs → DT ingest or OneAgent log collection runbook	`log_parsing_transformer` + runbook (Phase 11)
`drops`	NR drop rules → OpenPipeline filter processors	`drop_rule_transformer` (Phase 11)
`parsing`	NR Grok parsing → OpenPipeline DPL `parse` processors	`log_parsing_transformer` (Phase 11)

Component	Migrates	Transformer
`log_obfuscation`	NR PII / PAN masking → OpenPipeline `mask` processors (7 presets + regex)	`log_obfuscation_transformer` (Phase 17)
`log_archive`	NR Log Live Archive → Grail bucket + OpenPipeline egress (S3/GCS/Azure Blob)	`log_archive_transformer` (Phase 24)
`notifications`	NR Notification Channels → Workflow action tasks (email/slack/pagerduty/webhook first-class; Jira/ServiceNow/OpsGenie/xMatters/VictorOps/Teams via `http-function` fallback or `--legacy` typed shapes)	folded into `alert_transformer` (Phase 11 + 17)
`identity`	NR Users / Teams / Roles / SAML → `builtin:iam.*` + SCIM runbook	`identity_transformer` (Phase 17)
`cloud_integrations`	AWS (16 services) / Azure (8 resources) / GCP (8 services)	`cloud_integration_transformer` (Phase 18)
`kubernetes`	NR K8s → DynaKube (full-stack / host-only) + Helm values	`kubernetes_transformer` (Phase 18)
`aiops`	NR AI Workflows + enrichments + decisions → DT automation + Dynatrace Intelligence detectors	`aiops_transformer` (Phase 18)
`vulnerability`	NR Vuln Mgmt → RVA alerting + per-CVE muting	`vulnerability_transformer` (Phase 18)
`npm`	SNMP devices + NetFlow collector (secrets redacted)	`npm_transformer` (Phase 18)
`ai_monitoring`	Model registry + inference NRQL→DQL mapping	`ai_monitoring_transformer` (Phase 18)
`prometheus`	NR Prometheus → DT scrape + OTLP remote-write + relabel filters	`prometheus_transformer` (Phase 18)
`otel_metrics`	NR OTLP metrics ingest → `builtin:otel.ingest.metrics` + collector YAML	`otel_metrics_transformer` (Phase 23)
`otel_collector`	NR OTel collector (traces + metrics + logs + 5 processor kinds)	`otel_collector_transformer` (Phase 24)
`statsd`	NR StatsD → ActiveGate `builtin:statsd.metrics` + tag-mapping translation	`statsd_transformer` (Phase 23)
`cloudwatch_metric_streams`	NR Firehose path → `builtin:aws.metric-streams` + Terraform snippet	`cloudwatch_metric_streams_transformer` (Phase 23)
`database_monitoring`	10 DB engines (MySQL / Postgres / MSSQL / Oracle / MongoDB / Redis / Cassandra / MariaDB / DB2 / HANA) → `builtin:dynatrace.extension.db.*`	`database_monitoring_transformer` (Phase 24)
`on_host_integrations`	12 integrations (NGINX / HAProxy / Kafka / RabbitMQ / Elasticsearch / Memcached / Couchbase / Consul / Apache / etcd / Varnish / Zookeeper) → DT extensions	`on_host_integration_transformer` (Phase 24)
`security_signals`	NR Security Signals / IAST → AppSec envelope + per-signature OpenPipeline enrichment	`security_signals_transformer` (Phase 24)
`custom_entity`	NR custom entities → DT custom-device POST payload + enrichment matcher	`custom_entity_transformer` (Phase 24)
`metric_normalization`	NR rename / aggregate / drop rules → OpenPipeline metric processors	`metric_normalization_transformer` (Phase 24)
`saved_filter_notebook`	NR Data Apps → Document API `type=='notebook'` with markdown + DQL cells	`saved_filter_notebook_transformer` (Phase 24)

Recommended order: `workloads` first (foundations), then `dashboards`, then `synthetics`, then `alerts` (with `notification_channels`), then `slos`. Tag rules, log parsing, drop rules and the specialized surfaces in the table (cloud integrations, Kubernetes, Prometheus, database and on-host integrations, vulnerabilities, security signals, AI monitoring, OpenTelemetry, StatsD, CloudWatch metric streams, NPM, custom entities, metric normalization, saved-filter notebooks) are migrated by hand from the transformer guidance until `migrate` exports them.

5. Export Formats — Monaco & Terraform

Beyond direct API import, `Dynatrace-NewRelic` can emit **config-as-code** in two formats:

Monaco YAML

```
python3 migrate.py export-monaco --input ./output --output ./monaco-project
```

Output: a Monaco v2 project (`manifest.yaml` + per-config-type directories with `config.yaml` + JSON templates), including Platform SLOs as Monaco `slo-v2` configs. Deploy it with `monaco deploy manifest.yaml`. The SaaS Upgrade Assistant (M2S series) is not a route for this project: it imports a Dynatrace Managed configuration export created in the Cluster Management Console, not a Monaco project.

Terraform HCL

```
python3 migrate.py export-terraform --input ./output --output ./terraform-project
```

Output: HCL files using the `dynatrace-oss/dynatrace` provider, plus a `provider.tf` that declares the credential variables (`dynatrace_env_url` , `dynatrace_api_token` , `dynatrace_oauth_client_id` , `dynatrace_oauth_client_secret`). Resources: `dynatrace_document` (dashboards), `dynatrace_automation_workflow` , `dynatrace_segment` , `dynatrace_iam_policy` , `dynatrace_platform_slo` (SLOs), and `dynatrace_generic_setting` for anomaly detectors (`builtin:davis.anomaly-detectors`), synthetic tests and OpenPipeline processors (exporter on the `main` branch, read 10/14/2026). `--legacy` emits classic (Gen2) resource names instead.

Why both formats

- **Monaco** is best for one-time bootstrap and bulk deploys (`monaco deploy`)
- **Terraform** is best for ongoing GitOps-style management

The general guidance is **Terraform as the long-term tool**, with Monaco reserved for export and one-time deploys. The **AUTOM** series covers configuration automation tooling in depth.

6. End-to-End Runbook

This is the operational sequence to run a complete NR→DT migration.

Prerequisites

- [ ] NR API key + account ID configured
- [ ] DT Platform Token configured (OAuth / Classic tokens supported but Platform Token is the recommended default)
- [ ] Bucket strategy in place (see **ORGNZ-02 / ORGNZ-99**)
- [ ] Host group strategy in place (see **FAQ-01**)
- [ ] Stakeholders identified per wave

Phase 0 — Preflight (new in Phase 14; always do this first)

```
python3 migrate.py preflight
```

Probes the target tenant for Settings 2.0 / Document API / Automation API availability. If any Gen3 surface is missing, the command suggests `--legacy` and exits non-zero — catch this before burning a run.

Phase 1 — Discover (pair with NR2DT-01)

```
# Export from New Relic (nothing is written to Dynatrace)
python3 migrate.py migrate --export-only --output ./inventory
# Transform and write the conversion report without importing
python3 migrate.py migrate --dry-run --report --output ./inventory
```

Do not run `migrate --report` without `--dry-run`: with neither `--export-only` nor `--import-only`, `migrate` runs export, transform **and import**. Review `inventory/exports/newrelic_export.json`, enriched conversion-quality report (`confidence_score`, `warning_codes`, `runbook_url` per entry), and gap analysis. Stakeholder sign-off on wave plan.

Pre-decommission archive (Phase 17):

```
python3 migrate.py archive --account &lt;nr-account-id&gt; --since '30 days ago' \
  --event-types Transaction,Log,SyntheticCheck --output ./nrdb-archive
```

Historical NRDB data is **not migratable to Grail** — archive as JSONL before decommissioning NR. Resumable per-event-type cursors let you interrupt and restart safely.

Phase 2 — Translate & Stage

```
# Translate NRQL inventory
# Extract NRQL from the export (produces inventory/all-nrql.txt + .csv)
python3 migrate.py extract-nrql --input ./inventory --output ./inventory/all-nrql.txt
python3 migrate.py extract-nrql --input ./inventory --output ./inventory/all-nrql.csv

# Translate — compile for plain DQL; batch for a CSV with confidence scores
python3 migrate.py compile --file inventory/all-nrql.txt --output translated.dql
python3 migrate.py batch   --file inventory/all-nrql.csv --output translated.csv

# Instrumentation scan for newrelic.*() SDK calls (Phase 16) — one file per run
python3 migrate.py scan-instrumentation --file ./app-source/checkout/Handler.java --output ./instrumentation-todos.diff

# APM agent plan per language (Phase 16)
python3 migrate.py agents --language java --dry-run
python3 migrate.py agents --language nodejs --dry-run

# Transform each component and diff against the live tenant (nothing is imported)
python3 migrate.py migrate --dry-run --diff --report --components workloads --output ./wave0
```

Phase 3 — Wave-by-Wave Import (Gen3 default)

Use `migrate --dry-run --diff` before each wave and `--canary` for production tenants. Each wave below is a full run (export → transform → import), which is what writes `<output>/rollback-manifest.json`; `--diff` is ignored with `--import-only`.

```
# Wave 0 (foundations)
python3 migrate.py migrate --dry-run --diff --components workloads --output ./wave0
python3 migrate.py migrate --components workloads --canary 10 --canary-auto-proceed --output ./wave0

# Wave 1 (dashboards — read-only impact)
python3 migrate.py migrate --dry-run --diff --components dashboards --output ./wave1
python3 migrate.py migrate --components dashboards --output ./wave1

# Wave 2 (synthetics)
python3 migrate.py migrate --components synthetics --output ./wave2

# Wave 3 (alerts + notification channels — dual-alert window!)
python3 migrate.py migrate --components alerts,notification_channels --output ./wave3
# ... wait 1–2 weeks dual-alert ...

# Wave 4 (SLOs — also pulls in alerts and notification channels as dependencies)
python3 migrate.py migrate --components slos --output ./wave4
python3 migrate.py audit-slos

# Waves 5–6 (tags, logs, drop and parsing rules, specialized surfaces):
# not exported by `migrate` on the current main branch — migrate by hand (NRLC-07, §4)
```

Phase 4 — Validate & Cutover (NRLC-08)

Per-wave validation gates:

```
# Config-drift audit: live tenant vs the wave's transformed baseline (Phase 20)
python3 migrate.py audit --baseline ./wave1/transformed/dynatrace_config.json

# SLO validity audit (DT side only: SLOs evaluate, metrics exist)
python3 migrate.py audit-slos
```

Behavioral validation — the same query run in New Relic and in Dynatrace over the same window, results compared — has no CLI command; do it by hand per NRLC-08 §4. Sign-off per wave.

Phase 5 — Decommission NR

Disable NR alert routes; archive dashboards; halt NR ingest. Retain rollback manifests for 30+ days. `DynatraceClient.delete_entity` (Phase 20) now dispatches unified Gen3 deletes — rollback actually executes.

7. CI/CD Integration Patterns

Once the migration is run, ongoing config can live in CI:

GitOps with Terraform

```
# .github/workflows/dt-config.yml
name: dt-config
on: [push]
jobs:
  apply:
    runs-on: ubuntu-latest
    # Job-level env: plan and apply both need credentials.
    env:
      # Variables declared in the exported provider.tf
      TF_VAR_dynatrace_env_url: ${{ secrets.DT_ENV_URL }}
      TF_VAR_dynatrace_api_token: ${{ secrets.DT_API_TOKEN }}
      TF_VAR_dynatrace_oauth_client_id: ${{ secrets.DT_CLIENT_ID }}
      TF_VAR_dynatrace_oauth_client_secret: ${{ secrets.DT_CLIENT_SECRET }}
      # dynatrace_platform_slo reads its OAuth client from these
      DT_CLIENT_ID: ${{ secrets.DT_CLIENT_ID }}
      DT_CLIENT_SECRET: ${{ secrets.DT_CLIENT_SECRET }}
      DT_ACCOUNT_ID: ${{ secrets.DT_ACCOUNT_ID }}
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform init
      - run: terraform plan -out=tfplan
      - run: terraform apply tfplan
```

The `TF_VAR_*` names match the variables in the `provider.tf` that `export-terraform` writes; if you replace that provider block with your own, use the provider's environment variables instead (`DYNATRACE_ENV_URL` , `DYNATRACE_API_TOKEN` , and `DT_CLIENT_ID` / `DT_CLIENT_SECRET` / `DT_ACCOUNT_ID` or `DYNATRACE_PLATFORM_TOKEN`). The secret names on the right are placeholders.

> **Sources:** Dynatrace provider — authentication (Dynatrace GitHub) — *"Define `DYNATRACE_ENV_URL` for the Dynatrace environment URL."*, dynatrace_platform_slo resource (Dynatrace GitHub) — *"please define the environment variables `DT_CLIENT_ID` , `DT_CLIENT_SECRET` , `DT_ACCOUNT_ID` "*, Monaco commands (DT docs), SaaS Upgrade Assistant (DT docs).

Translation in CI

Run the compiler in CI to validate any committed DQL:

```
      - run: npx nrql-translator validate --file ./queries.nrql
      - run: |
          if [[ $(jq '.confidence' result.json) == "LOW" ]]; then
            echo "::error::LOW-confidence translation — manual review required"
            exit 1
          fi
```

Drift Detection

Schedule a daily drift audit to detect manual DT changes that drifted from the GitOps source of truth — Phase 20 `migrate.py audit` :

```
# audit exits 1 when it finds drift; the JSON report carries drift_count and by_kind
if ! python3 migrate.py audit --baseline ./output/transformed/dynatrace_config.json --output drift.json; then
  alert_team "DT config drift detected"
fi
```

Summary

The toolchain is open-source, modular, and CI-friendly. `Dynatrace-NewRelic` orchestrates; `nrql-engine` translates; `nrql-translator` exposes the engine on the CLI. Together they cover discovery, translation, transformation, validation, drift audit, canary promotion, and rollback. Pair them with the bucket strategy (**ORGNZ-02 / ORGNZ-99**) and host grouping strategy (**FAQ-01**) for a complete migration foundation.

Series Complete

You've reached the end of the NRLC deep-dive series. For the procedural step-by-step migration walkthrough, see the **NR2DT** series (9 procedural steps + NR2DT-99 summary). For reference, NRLC covers:

#	Topic
01	Platform Comparison
02	NRQL → DQL Translation
03	Dashboard Migration
04	Alert & Workflow Migration
05	Synthetic Monitor Migration
06	SLO & Workload Migration
07	Logs, Tags & Drop Rules
08	Validation, Diff & Rollback
09	Toolchain Reference & End-to-End Runbook

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source NewRelic-to-Dynatrace-Migration-Utilities, nrql-engine (planned future home: the `dynatrace-dma` Dynatrace Migration Assistant organization), and nrql-translator projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official Dynatrace documentation and New Relic documentation.